

👁 Watch 0 ▾

🍴 Fork 0 ▾

★

A configuration framework that enhances Claude Code with specialized commands, cognitive personas, and development methodologies.

-  MIT License
-  [superclaude.netlify.app/](#)
-  Code of conduct
-  Contributing
-  Security policy
- ☆ 1 star 🍴 0 forks 👁 0 watching 🌿 2 branches 🏷 0 tags ↗ Activity
- 🌐 Public repository · Forked from [SuperClaude-Org/SuperClaude_Framework](#)

🌿 m ▾

🌿 2 Branches 🏷 0 Tags 🌿

Q Go to file

T

Go to file

Add file ▸



Code

⋮

This branch is **1 commit behind** SuperClaude-Org/SuperClaude_Framework:master .

 **Contribute** ▾

 **Sync fork** ▾

 **jakeefr** Add Windows installation guide ([SuperClaude-Org#553](#)) 

226c45c · 3 months ago 

 MANIFEST.in	chore: bump version to 4.1.9 and add __init_...	8 months ago
 Makefile	Update Makefile and add uninstall script	8 months ago
 PARALLEL_INDEXING_PLAN.md	Proposal: Create next Branch for Testing Gr...	9 months ago
 PLANNING.md	fix: fill implementation gaps across core mo...	4 months ago
 PLUGIN_INSTALL.md	Proposal: Create next Branch for Testing Gr...	9 months ago
 PROJECT_INDEX.json	Proposal: Create next Branch for Testing Gr...	9 months ago
 PROJECT_INDEX.md	Proposal: Create next Branch for Testing Gr...	9 months ago
 PR_DOCUMENTATION.md	Proposal: Create next Branch for Testing Gr...	9 months ago
 QUALITY_COMPARISON.md	Proposal: Create next Branch for Testing Gr...	9 months ago
 README-ja.md	fix: fill implementation gaps across core mo...	4 months ago
 README-kr.md	fix: fill implementation gaps across core mo...	4 months ago
 README-zh.md	fix: fill implementation gaps across core mo...	4 months ago
 README.md	fix: fill implementation gaps across core mo...	4 months ago
 SECURITY.md	refactor: PEP8 compliance - directory renam...	9 months ago
 TASK.md	fix: fill implementation gaps across core mo...	4 months ago
 TEST_PLUGIN.md	Proposal: Create next Branch for Testing Gr...	9 months ago
 VERSION	fix: fill implementation gaps across core mo...	4 months ago
 install.sh	Add missing install.sh script (SuperClaude-...)	8 months ago
 package.json	fix: fill implementation gaps across core mo...	4 months ago
 pyproject.toml	fix: fill implementation gaps across core mo...	4 months ago
 setup.py	Add missing install.sh script (SuperClaude-...)	8 months ago

README

Code of conduct

Contributing

License

Security



SuperClaude Framework

Uses

▲ 1

Transform Claude Code into a Structured Development Platform

 mentioned in awesome

Try

SuperGemini Framework

Try

SuperQwen Framework

version

4.3.0

 Tests

failing

License

MIT

PRs

welcome

Visit Website

pypi

v4.3.0

downloads

370k

npm

v4.0.7

 English

 中文

 日本語

[Quick Start](#) • [Support](#) • [Features](#) • [Docs](#) • [Contributing](#)



Framework Statistics

Commands	Agents	Modes	MCP Servers
30	20	7	8
Slash Commands	Specialized AI	Behavioral	Integrations

30 slash commands covering the complete development lifecycle from brainstorming to deployment.



Overview

SuperClaude is a **meta-programming configuration framework** that transforms Claude Code into a structured development platform through behavioral instruction injection and component orchestration. It provides systematic workflow automation with powerful tools and intelligent agents.



Disclaimer

This project is not affiliated with or endorsed by Anthropic. Claude Code is a product built and maintained by [Anthropic](#).



For Developers & Contributors

Essential documentation for working with SuperClaude Framework:

Document	Purpose	When to Read
PLANNING.md	Architecture, design principles, absolute rules	Session start, before implementation
TASK.md	Current tasks, priorities, backlog	Daily, before starting work
KNOWLEDGE.md	Accumulated insights, best practices, troubleshooting	When encountering issues, learning patterns
CONTRIBUTING.md	Contribution guidelines, workflow	Before submitting PRs
Commands Reference	Complete reference for all 30 <code>/sc:*</code> commands with syntax, examples, workflows, and decision guides	Learning SuperClaude, choosing the right command

Pro Tip: Claude Code reads these files at session start to ensure consistent, high-quality development aligned with project standards.



New to SuperClaude? Start with [Commands Reference](#) – it contains visual decision trees, detailed command comparisons, and workflow examples to help you understand which commands to use and when.



Quick Installation

IMPORTANT: The TypeScript plugin system described in older documentation is not yet available (planned for v5.0). For current installation instructions, please follow the steps below for v4.x.



Current Stable Version (v4.3.0)

SuperClaude currently uses slash commands.

Option 1: pipx (Recommended)

```
# Install from PyPI
pipx install superclaude
```

```
# Install commands (installs all 30 slash commands)
superclaude install
```

```
# Install MCP servers (optional, for enhanced capabilities)
superclaude mcp --list      # List available MCP servers
superclaude mcp             # Interactive installation
```



```
superclaude mcp --servers tavily --servers context7 # Install specific servers
```

```
# Verify installation
superclaude install --list
superclaude doctor
```

After installation, restart Claude Code to use 30 commands including:

- `/sc:research` - Deep web research (enhanced with Tavily MCP)
- `/sc:brainstorm` - Structured brainstorming
- `/sc:implement` - Code implementation
- `/sc:test` - Testing workflows
- `/sc:pm` - Project management
- `/sc` - Show all 30 available commands

Option 2: Direct Installation from Git

```
# Clone the repository
git clone https://github.com/SuperClaude-Org/SuperClaude_Framework.git
cd SuperClaude_Framework

# Run the installation script
./install.sh
```



Coming in v5.0 (In Development)

We are actively working on a new TypeScript plugin system (see issue [#419](#) for details). When released, installation will be simplified to:

```
# This feature is not yet available
/plugin marketplace add SuperClaude-Org/superclaude-plugin-marketplace
/plugin install superclaude
```



Status: In development. No ETA has been set.



Enhanced Performance (Optional MCPs)

For **2-3x faster** execution and **30-50% fewer tokens**, optionally install MCP servers:

```
# Optional MCP servers for enhanced performance (via airis-mcp-gateway):
# - Serena: Code understanding (2-3x faster)
# - Sequential: Token-efficient reasoning (30-50% fewer tokens)
# - Tavily: Web search for Deep Research
# - Context7: Official documentation lookup
# - Mindbase: Semantic search across all conversations (optional enhancement)

# Note: Error learning available via built-in ReflexionMemory (no installation required)
# Mindbase provides semantic search enhancement (requires "recommended" profile)
# Install MCP servers: https://github.com/agiletec-inc/airis-mcp-gateway
# See docs/mcp/mcp-integration-policy.md for details
```



Performance Comparison:

- **Without MCPs:** Fully functional, standard performance ✅
- **With MCPs:** 2-3x faster, 30-50% fewer tokens ⚡



Support the Project

Hey, let's be real - maintaining SuperClaude takes time and resources.

The Claude Max subscription alone runs \$100/month for testing, and that's before counting the hours spent on documentation, bug fixes, and feature development. If you're finding value in SuperClaude for your daily work, consider supporting the project. Even a few dollars helps cover the basics and keeps development active.

Every contributor matters, whether through code, feedback, or support. Thanks for being part of this community! 🙏

**Ko-fi**

One-time contributions

**Patreon**

Monthly support

**GitHub**

Flexible tiers

Your Support Enables:

Item	Cost/Impact
 Claude Max Testing	\$100/month for validation & testing
 Feature Development	New capabilities & improvements
 Documentation	Comprehensive guides & examples
 Community Support	Quick issue responses & help
 MCP Integration	Testing new server connections
 Infrastructure	Hosting & deployment costs

Note: No pressure though - the framework stays open source regardless. Just knowing people use and appreciate it is motivating. Contributing code, documentation, or spreading the word helps too! 🙏

 What's New in v4.1

Version 4.1 focuses on stabilizing the slash command architecture, enhancing agent capabilities, and improving documentation.

**Smarter Agent System**
20 specialized agents with domain expertise:

- PM Agent ensures continuous learning through systematic documentation
- Deep Research agent for autonomous web research
- Security engineer catches real vulnerabilities
- Frontend architect understands UI patterns
- Automatic coordination based on context
- Domain-specific expertise on demand

**Optimized Performance**
Smaller framework, bigger projects:

- Reduced framework footprint
- More context for your code
- Longer conversations possible
- Complex operations enabled

 **MCP Server Integration**

8 powerful servers with easy CLI installation:

```
# List available MCP servers
superclaude mcp --list

# Install specific servers
superclaude mcp --servers tavily context7

# Interactive installation
superclaude mcp
```

Available servers:

- **Tavily** → Primary web search (Deep Research)
- **Context7** → Official documentation lookup
- **Sequential-Thinking** → Multi-step reasoning
- **Serena** → Session persistence & memory
- **Playwright** → Cross-browser automation
- **Magic** → UI component generation
- **Morphllm-Fast-Apply** → Context-aware code modifications
- **Chrome DevTools** → Performance analysis

 **Behavioral Modes**

7 adaptive modes for different contexts:

- **Brainstorming** → Asks right questions
- **Business Panel** → Multi-expert strategic analysis
- **Deep Research** → Autonomous web research
- **Orchestration** → Efficient tool coordination
- **Token-Efficiency** → 30-50% context savings
- **Task Management** → Systematic organization
- **Introspection** → Meta-cognitive analysis

 **Documentation Overhaul**

Complete rewrite for developers:

- Real examples & use cases
- Common pitfalls documented
- Practical workflows included
- Better navigation structure

 **Enhanced Stability**

Focus on reliability:

- Bug fixes for core commands
- Improved test coverage
- More robust error handling
- CI/CD pipeline improvements

 Deep Research Capabilities

 Autonomous Web Research Aligned with DR Agent Architecture

SuperClaude v4.2 introduces comprehensive Deep Research capabilities, enabling autonomous, adaptive, and intelligent web research.

 **Adaptive Planning**

Three intelligent strategies:

- **Planning-Only**: Direct execution for clear queries
- **Intent-Planning**: Clarification for ambiguous requests
- **Unified**: Collaborative plan refinement (default)

 **Multi-Hop Reasoning**

Up to 5 iterative searches:

- Entity expansion (Paper → Authors → Works)
- Concept deepening (Topic → Details → Examples)
- Temporal progression (Current → Historical)
- Causal chains (Effect → Cause → Prevention)

 **Quality Scoring**

Confidence-based validation:

- Source credibility assessment (0.0-1.0)
- Coverage completeness tracking
- Synthesis coherence evaluation

 **Case-Based Learning**

Cross-session intelligence:

- Pattern recognition and reuse
- Strategy optimization over time
- Successful query formulations saved

- Minimum threshold: 0.6, Target: 0.8
- Performance improvement tracking

Research Command Usage

```
# Basic research with automatic depth
/research "latest AI developments 2024"

# Controlled research depth (via options in TypeScript)
/research "quantum computing breakthroughs" # depth: exhaustive

# Specific strategy selection
/research "market analysis" # strategy: planning-only

# Domain-filtered research (Tavily MCP integration)
/research "React patterns" # domains: reactjs.org,github.com
```

Research Depth Levels

Depth	Sources	Hops	Time	Best For
Quick	5-10	1	~2min	Quick facts, simple queries
Standard	10-20	3	~5min	General research (default)
Deep	20-40	4	~8min	Comprehensive analysis
Exhaustive	40+	5	~10min	Academic-level research

Integrated Tool Orchestration

The Deep Research system intelligently coordinates multiple tools:

- Tavily MCP: Primary web search and discovery
- Playwright MCP: Complex content extraction
- Sequential MCP: Multi-step reasoning and synthesis
- Serena MCP: Memory and learning persistence
- Context7 MCP: Technical documentation lookup

Documentation

Complete Guide to SuperClaude

 Getting Started	 User Guides	 Developer Resources	 Reference
 Quick Start Guide Get up and running fast  Installation Guide Detailed setup instructions	 Slash Commands All 30 commands organized by category  Agents Guide 20 specialized agents  Behavioral Modes 7 adaptive modes  Flags Guide Control behaviors  MCP Servers 8 server integrations	 Technical Architecture System design details  Contributing Code Development workflow  Testing & Debugging Quality assurance	 [**Examples Cookbook**] (docs/reference/examples-cookbook.md) *Real-world recipes*  Troubleshooting Common issues & fixes

-  [Session Management](#)
Save & restore state



Contributing



Join the SuperClaude Community

We welcome contributions of all kinds! Here's how you can help:

Priority	Area	Description
 High	Documentation	Improve guides, add examples, fix typos
 High	MCP Integration	Add server configs, test integrations
 Medium	Workflows	Create command patterns & recipes
 Medium	Testing	Add tests, validate features
 Low	i18n	Translate docs to other languages

 Read [Contributing Guide](#)  View [All Contributors](#)



License

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors

No contributors

Languages

Python 81.7% **TypeScript** 10.1% **Shell** 6.6% **Makefile** 1.6%

Suggested workflows

Based on your tech stack



Python application

Create and test a Python application.
By GitHub Actions

Configure



Python package

Create and test a Python package on multiple Python versions.
By GitHub Actions

Configure



SLSA Generic generator

Generate SLSA3 provenance for your existing release workflows
By Open Source Security Foundation (OpenSSF)

Configure

[More workflows](#)



